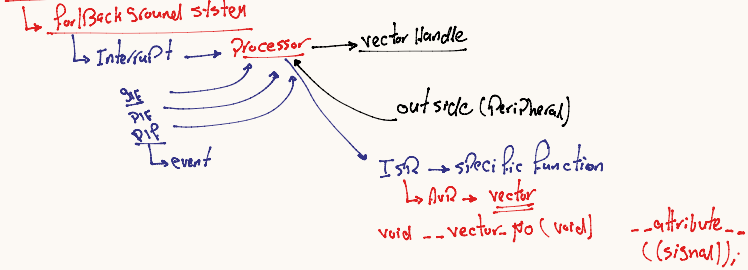
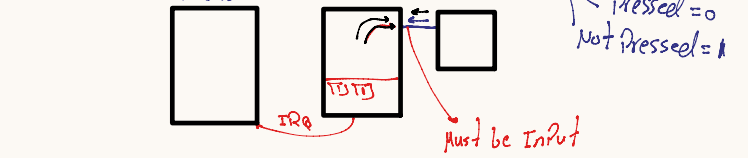


Revision



Peripheral → EXTI
Need → we need H.W component outside Mc will be Interrupt the Processor

EXTI → it's Peripheral Can be Interrupt Processor Based on H.W Component Event



* what event happen to this pin?
 ↳ level → ① low level
 ↳ level → ② High level
 ↳ edge → ① rising
 ↳ edge → ② Falling
 ↳ edge → ③ on change

You can configure the event type will generate Interrupt in EXTI

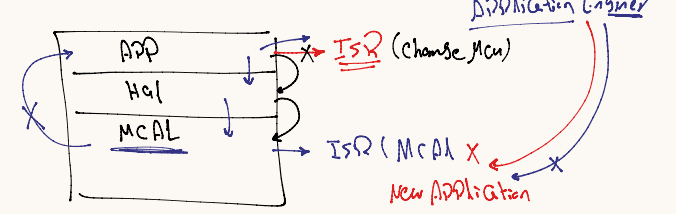
EXTI → in side Atmega 32

Processor	EXTI	Extio	Exti1	Exti2
MCUCR	Configuration event Exti0, Exti1	MCUCR	Configuration event Exti0, Exti1	MCUCR
MCUCSR	Configuration event Exti2	MCUCSR	Configuration event Exti2	MCUCSR
GICR	PIF →	GICR	PIF →	GICR
GIFR	PIF →	GIFR	PIF →	GIFR

Exti0	Exti1	Exti2
① event type Config 131	① event type Config 131	① event type Config 131
MCUCR →	MCUCR →	MCUCR →
ISCR1	ISCR1	ISCR1
ISCR0	ISCR0	ISCR0
low	low	low
high	high	high
rising	rising	rising
falling	falling	falling
on change	on change	on change
② Pin number	② Pin number	② Pin number
Group 0, 2 → PD2	Group 0, 3 → PD3	Group 0, 2 → PD2
③ PIF →	③ PIF →	③ PIF →
GICR → 6 → INT0	GICR → 7 → INT1	GICR → 5 → INT2
④ PIF →	④ PIF →	④ PIF →
GIFR → 6 → INTF0	GIFR → 7 → INTF1	GIFR → 5 → INTF2
⑤ vector no	⑤ vector no	⑤ vector no
①	②	③

* Where write the ISR?

- * ① content → ISR → Change → Based APP (ISR → APP)
- * ② ISR work → MCU (ISR → MCU)



* Call Back Function

- ① ISR Based on MCU → write in Drive in MCAL
- ② Provide function to take the new Application.

→ Note ∴ Pointer to Function
 Return (*Pointer Name) (Argument)
 int Add(int x, int y);
 int (*PFAdd)(int, int) = Add;
 PFAdd(5, 7);

Driver (Exti)

```
void (*global)(void) = Null;

void CallBack(void (*PF)(void))
{
    "PF" → ledApp
    global = PF; // global = PF = ledApp
}

void __vector_1(void) __attribute__((signal));

void __vector_1(void) → ISR

global(); ←

Application (Main.c)

void ledApp()
{
    // ...
}

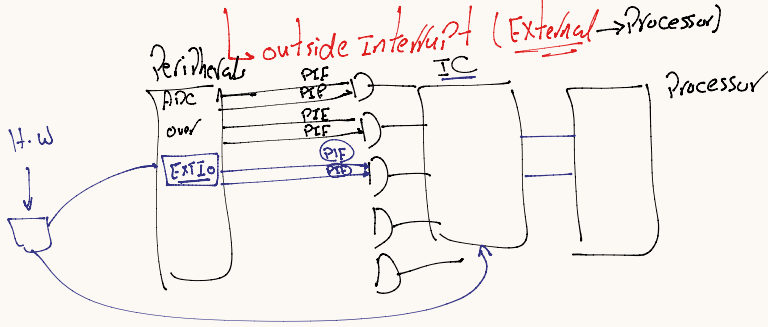
void Main()
{
    CallBack(ledApp);
}
```

Interrupt type

① Position

↳ inside Interrupt (internal → Processor)

↳ outside Interrupt (External → Processor)



Bit wise operation

Reg →



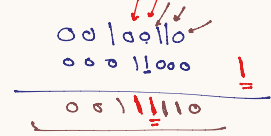
Reg = 00 00 0110

mask = 1100 1111

mask → No → Bit Interested = 1
Bit Not Interested = 0

to clear multiBit → $Reg \& \sim Mask$

Reg =



set →

Reg | Mask

Reg → 00011001

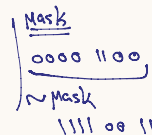
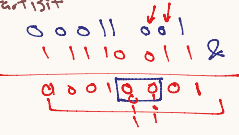
Bit 2, 3

Bit Mask Rule

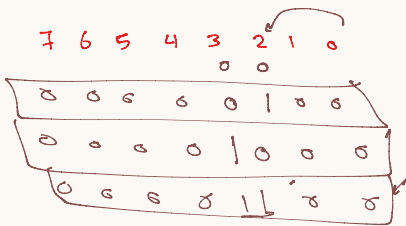
$Reg = (Reg \& \sim Mask) | (Value \ll 2)$

low → 0
Analogic → 1
Polling → 2
Rising → 3

$Reg = (Reg \& \sim Mask) | Value \ll startBit$



- low level
- ① ← analogic
 - ② ← Polling
 - ③ ← Rising



$3 \ll 2$

0000 0011 $\ll 2$

0000 1100